# Thesis Plan

## Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien
University of Helsinki, Master's Programme in Computer Science (Software study track)
Thesis supervisor: Keijo Heljanko
Nokia instructors: Christoph Luneburg, Petteri Kotimaa

13 August 2026

Estimated completion: 11 November 2026 Planned thesis presentation: November 2026

## 1 Background and motivation

Modern RAN control-plane software (the L3 of a gNB, e.g. the gNB-CU-CP [1]) is written mostly in C++, which delivers the throughput and latency such systems need but carries well-known memory-safety and concurrency risks (use-after-free, data races, double-free). These classes dominate real defect populations, and migrating to memory-safe languages is now an explicit policy position [2, 3]. Rust offers comparable performance with safety guaranteed at compile time on machine-checked foundations [4].

Its fit for latency-critical telecom control planes is unproven. The "Rust performs like C++" consensus rests on general-purpose benchmarks [5] that report average-case behaviour. Closer studies find real costs, both in networking code [6] and in specific mechanisms such as bounds and overflow checking [7]. For a RAN control plane the operative metric is not the average but the tail [8]. This thesis is carried out at Nokia on a subsystem modelled on a real Nokia gNB-CU-CP component.

The decision has a long horizon. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, with full specifications due to the ITU by mid-2030 and migration from 5G explicitly in scope [9, 10], so a language chosen for the control plane of the current generation is inherited by the next.

## 2 Research gap

The closest prior work is a cluster of at least five master's theses carried out at Ericsson. Karlbäck and Orö compare Rust and Go against C++ for RAN applications and conclude C++ remains the choice where performance is critical [11]. Olsson reaches the same domain from the management side and reports Rust's execution speed on par with or better than C's, but its peak memory higher [12]. Olsson and Stenström build a thread-per-core packet processor on asynchronous Rust and measure between 20% and 50% of the throughput of the DPDK application it mirrors [13]. Arvidsson and Kaller measure median and 99th-percentile latency for 5G control-plane procedures across five asynchronous frameworks: C++ with Boost.Fibers is fastest at every load. The Rust runtimes track the shape of its median curve, but their p99 rises more steeply past a threshold rate [14]. Emberg and Wadsten come closest in method, building matched C++ and Rust ASN.1 decompilers, reaching third-party C++ from Rust through a C wrapper, and comparing the two statistically [15].

Three of the five vary the runtime and concurrency model together with the language: Boost.Fibers, goroutines, Tokio, Monoio, Glommio, DPDK Eventdev. A measured difference therefore cannot be attributed to the language rather than to the framework built on it, and the effect sizes involved, up to a factor of five, indicate how much of the measurement the runtime owns. Independent real-time analysis of those runtimes supports that reading [16]. The remaining two report execution time, and in one case peak memory, rather than tail behaviour under rising load.

Stated as requirements rather than as an absence, the gap is narrow, and each requirement is met somewhere. Matched implementations built from one design exist in systems software [6, 17, 18, 19] and, in telecom, in Emberg and Wadsten; percentile latency for control-plane procedures is reported by Arvidsson and Kaller; adoption effort is measured by Karlbäck and Orö. No study combines them for a stateful, concurrent control-plane subsystem: holding the specification, algorithms, concurrency model, allocator, and build settings fixed, *and* sweeping past the saturation knee, *and* measuring what incremental migration costs. All five theses are also unindexed grey literature, absent from the citation databases and reachable only through repository search, so the evidence a vendor would weigh this decision on has not been through review.

Two further properties of that evidence matter. It is concentrated in one vendor: the five studies use that vendor's frameworks and hardware targets, and none of their conclusions has been tested against another organisation's software, so their transferability is untested rather than established. And the record on the other side is emptier still. A search of Nokia's indexed publications returns no study of Rust, and none on memory safety in the radio access network. The nearest peer-reviewed position from that side, co-authored by two Nokia researchers, argues for making memory-error sanitisation compilers practical on the premise that certifying compilation is not [20], which is the after-the-fact detection strategy this thesis sets a compile-time guarantee against. The question is therefore open in the literature and unexamined in this setting, and the artefacts published with the thesis let an independent organisation check the answer.

## 3 Objective and research questions

The thesis builds two functionally equivalent implementations of a UE Context Management service in modern C++ and in Rust and compares them under a spec-justified synthetic signalling workload. Since similar average throughput is already known, it targets four less-explored dimensions:

**RQ1** How do Rust and C++ compare in latency predictability (median, p99, p99.9, maximum) under increasing load and overload, with the concurrency model held fixed?

**RQ2** Which classes of C++ memory and concurrency defects does safe Rust eliminate at compile time, at what stage are the rest detected, and which classes does it *not* prevent?

**RQ3** What overhead, if any, does Rust's safety model impose on throughput, CPU, and memory?

**RQ4** What are the practical implications (effort, FFI interoperability, risk) of adopting Rust incrementally in an existing C++ RAN codebase?

RQ2 is deliberately two-sided: a catalogue containing only defects Rust prevents would confirm its own premise, so it spans the boundary: seven classes safe Rust rejects and seven it does not. Memory-safety defects in real Rust code invariably trace back to `unsafe` [21], which one boundary class now reproduces directly, while deadlock remains an open problem for the language [22]. Two of the seven boundary classes returned the opposite of the expected result, with the C++ tooling reporting what Rust's does not, which is the kind of finding a one-sided catalogue could not have produced.

## 4 Methodology

A single language-agnostic specification (public API, UE state machine, data structures, sharding and concurrency model) is frozen before any implementation code is written, so the comparison measures language rather than design differences. Both implementations use identical algorithms, a shared-nothing sharded worker-thread model, the same allocator, and comparable build optimisation. Holding the concurrency model fixed matters because thread-per-core structuring itself changes tail latency [23]. Both must pass the same parity suite, whose expected results come from an independent third implementation of the specification rather than from either language, so that agreement between the two cannot be mistaken for correctness of both.

The workload is swept from nominal load to overload on pinned cores, with interleaved repeats and interval estimates rather than single figures [24]: medians with nonparametric bootstrap confidence intervals [25], and exact paired permutation tests corrected across the family by the Holm procedure [26]. Frequency locking belongs to the bare-metal protocol. The virtualised host measured so far does not expose the governor from inside the guest. Latency is captured in high-dynamic-range histograms against an intended send schedule, so coordinated omission does not hide the tail [27, 28]. Evaluation adds a controlled fault-injection study [29] and an FFI prototype. On the FFI, the plan takes a narrow position: confining `unsafe` to the seam is a statement about what the compiler checks, not run-time protection, since host and component share an address space [30]. Only synthetic data is used, so no personal or confidential data is processed.

## 5 Expected contributions

- A reproducible, telecom-relevant benchmarking framework.
- A frozen shared specification and two parity-tested implementations.
- A quantitative comparison emphasising tail latency under overload.
- A defect-class taxonomy mapping both what the compile-time guarantee covers and where it stops.
- Evidence-based guidelines for incremental Rust adoption, where the dominant cost is the learning curve rather than run-time [31].

## 6 Status and timeline

Work is well under way. The shared specification, both implementations, the conformance suite, the fault-injection catalogue, the FFI prototype, and the benchmark harness are in place. Chapter drafts and a supervisor-read-through skeleton (`Thesis_v1.tex`) exist. The bare-metal measurement campaign has been run and resolves the RQ1 tail claim, though at a shard and repeat count of its own, so it corroborates the main sweep and does not replace it. What remains is a matched repeat of it, final chapter prose for Design through Conclusion, and submission.

| Item | State |
|---|---|
| Literature review | Drafted in thesis skeleton |
| Spec + implementations | Complete; awaiting supervisor freeze |
| RQ2 (fault injection) | Complete |
| RQ4 (FFI prototype) | Complete |
| RQ1 / RQ3 (benchmarks) | Measured on a dedicated virtualised Xeon host, with a bare-metal confirmation run resolving the far tail |
| Thesis document | In progress (full draft + bullet skeleton) |

| Month (2026) | Goal | What happens |
|---|---|---|
| June | Foundations | Done: topic defined; C++/Rust/5G foundations; literature review drafted. |
| July | Draft | Done: thesis plan approved; chapter skeleton and evaluation design outlined. |
| August | Build | In progress: specification and both implementations built; parity suite and benchmark harness operational; bare-metal campaign next. |
| September | Measure | Provision a bare-metal testbed; run the full campaign; generate figures and numbers. |
| October | Resolve | Finalise RQ1/RQ3 analysis; confirm or retract the tail claim; revise affected chapters; draft to supervisors and Nokia instructors. |
| November | Submit | Final revisions, colloquium presentation, submission. |

**Risks.** If the testbed is delayed, RQ1 and RQ3 may remain preliminary. A cloud bare-metal instance or a Nokia lab host is the fallback. If the tail difference proves unresolvable even on bare metal, it is reported as a bounded null result with the confidence interval stated, which is a legitimate finding. Single-author skill asymmetry is bounded by the frozen specification and parity suite, and acknowledged as residual for performance.

**Out of scope.** The full gNB, PHY/L2 and the user plane, real captured traces, managed-language alternatives, and production operational concerns.

## References

[1] 3GPP. *NG-RAN; Architecture description.* Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: `https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip`.

[2] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation.* BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: `https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL`.

[3] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously.* Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: `https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps`.

[4] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: `10.1145/3158154`. URL: `https://doi.org/10.1145/3158154`.

[5] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. “Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?” In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE).* 2017, pp. 256–267. DOI: `10.1145/3136014.3136031`. URL: `https://doi.org/10.1145/3136014.3136031`.

[6] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. “The Case for Writing Network Drivers in High-Level Programming Languages”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS).* 2019,

pp. 1–13. DOI: `10.1109/ANCS.2019.8901892`. URL: `https://doi.org/10.1109/ANCS.2019.8901892`.

[7] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. “Towards Understanding the Runtime Performance of Rust”. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE).* 2022, 140:1–140:6. DOI: `10.1145/3551349.3559494`. URL: `https://doi.org/10.1145/3551349.3559494`.

[8] J. Dean and L. A. Barroso. “The Tail at Scale”. In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: `10.1145/2408776.2408794`. URL: `https://doi.org/10.1145/2408776.2408794`.

[9] J. Almodovar and A. Sultan. *The SA1 road to 6G.* 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: `https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road`.

[10] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond.* Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 (“6G”) framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: `https://www.itu.int/rec/R-REC-M.2160/en`.

[11] R. Karlbäck and A. Orö. “Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2021. URL: `https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307`.

[12] V. Olsson. “Rust Programming Language as an Alternative to C for RAN Management Applications”. Carried out at Ericsson AB. Master’s thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: `https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822`.

[13] J. Olsson and I. Stenström. “Rust Async for Thread-per-Core Packet Processing”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: `https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029`.

[14] A. Arvidsson and M. Kaller. “Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2025. URL: `https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746`.

[15] C. Emberg and A. Wadsten. “Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages”. Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: `https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552`.

[16] M. Škoudlil, M. Sojka, and Z. Hanzálek. “A First Look at ROS 2 Applications Written in Asynchronous Rust”. In: *37th Euromicro Conference on Real-Time Systems (ECRTS 2025).* Vol. 335. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 1:1–1:21. DOI: `10.4230/LIPIcs.ECRTS.2025.1`. URL: `https://doi.org/10.4230/LIPIcs.ECRTS.2025.1`.

[17] Z. Li, T. Huang, V. Narayanan, and A. Burtsev. “Understanding the Overheads of Hardware and Language-Based IPC Mechanisms”. In: *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. 2021, pp. 53–61. DOI: `10.1145/3477113.3487275`. URL: `https://doi.org/10.1145/3477113.3487275`.

[18] F. Suchert and J. Castrillon. “STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks”. In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: `10.1007/978-3-031-31180-2_10`. URL: `https://doi.org/10.1007/978-3-031-31180-2_10`.

[19] A. Gonzalez, D. Mvondo, and Y.-D. Bromberg. “Takeaways of Implementing a Native Rust UDP Tunneling Network Driver in the Linux Kernel”. In: *Proceedings of the 12th Workshop on Programming Languages and Operating Systems (PLOS)*. 2023, pp. 18–25. DOI: `10.1145/3623759.3624547`. URL: `https://doi.org/10.1145/3623759.3624547`.

[20] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. “Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers”. In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: `10.1109/QRS.2016.44`. URL: `https://doi.org/10.1109/QRS.2016.44`.

[21] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. “Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs”. In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: `10.1145/3466642`. URL: `https://doi.org/10.1145/3466642`.

[22] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. “Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs”. In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: `10.1007/s10515-026-00614-z`. URL: `https://doi.org/10.1007/s10515-026-00614-z`.

[23] P. Enberg, A. Rao, and S. Tarkoma. “The Impact of Thread-per-Core Architecture on Application Tail Latency”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: `10.1109/ANCS.2019.8901874`. URL: `https://doi.org/10.1109/ANCS.2019.8901874`.

[24] A. Georges, D. Buytaert, and L. Eeckhout. “Statistically Rigorous Java Performance Evaluation”. In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: `10.1145/1297027.1297033`. URL: `https://doi.org/10.1145/1297027.1297033`.

[25] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[26] S. Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: `https://www.jstor.org/stable/4615733`.

[27] B. Schroeder, A. Wierman, and M. Harchol-Balter. “Open Versus Closed: A Cautionary Tale”. In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI ’06)*. 2006. URL: `https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale`.

[28] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section “Corrected vs. raw value recording calls” documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: `https://github.com/HdrHistogram/HdrHistogram`.

[29] R. Natella, D. Cotroneo, and H. S. Madeira. “Assessing Dependability with Software Fault Injection: A Survey”. In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: `10.1145/2841425`. URL: `https://doi.org/10.1145/2841425`.

[30] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. “Keeping Safe Rust Safe with Galeed”. In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: `10.1145/3485832.3485903`. URL: `https://doi.org/10.1145/3485832.3485903`.

[31] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. “Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study”. In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: `https://www.usenix.org/conference/soups2021/presentation/fulton`.